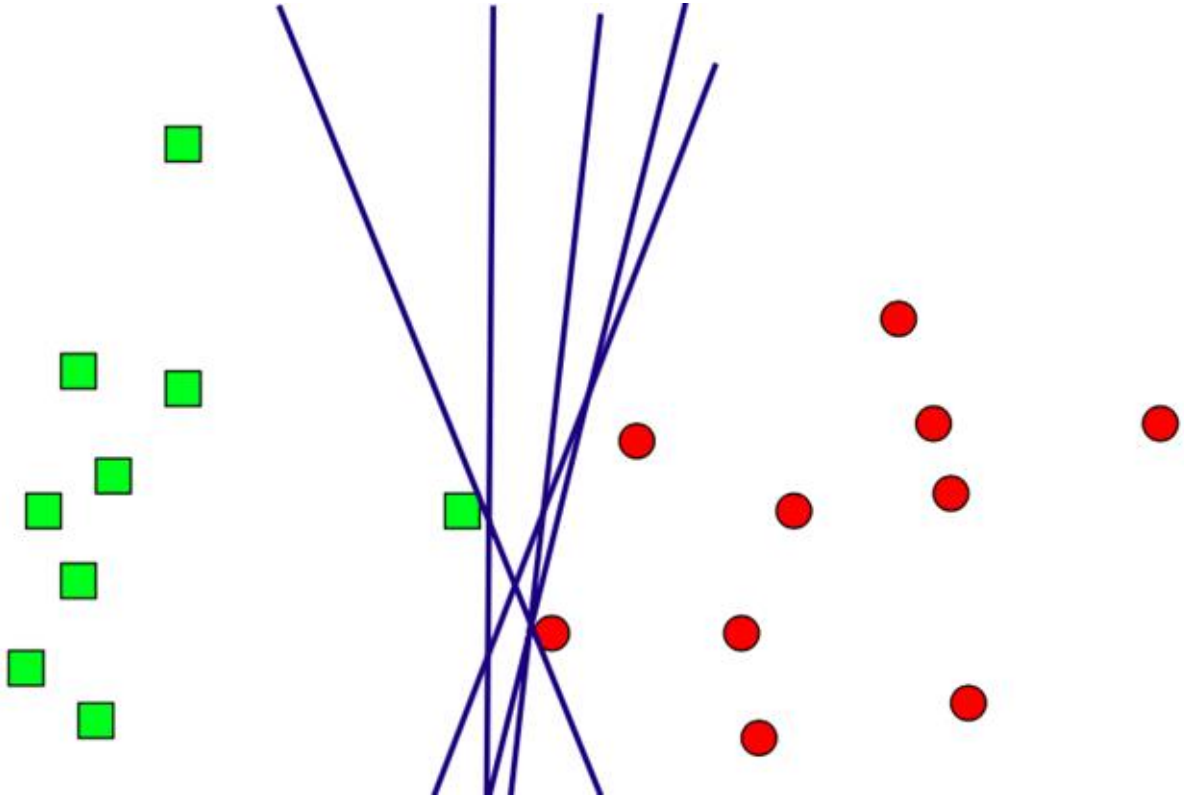
SUPPORT VECTOR MACHINES

Many slides courtesy of Marios Savvides

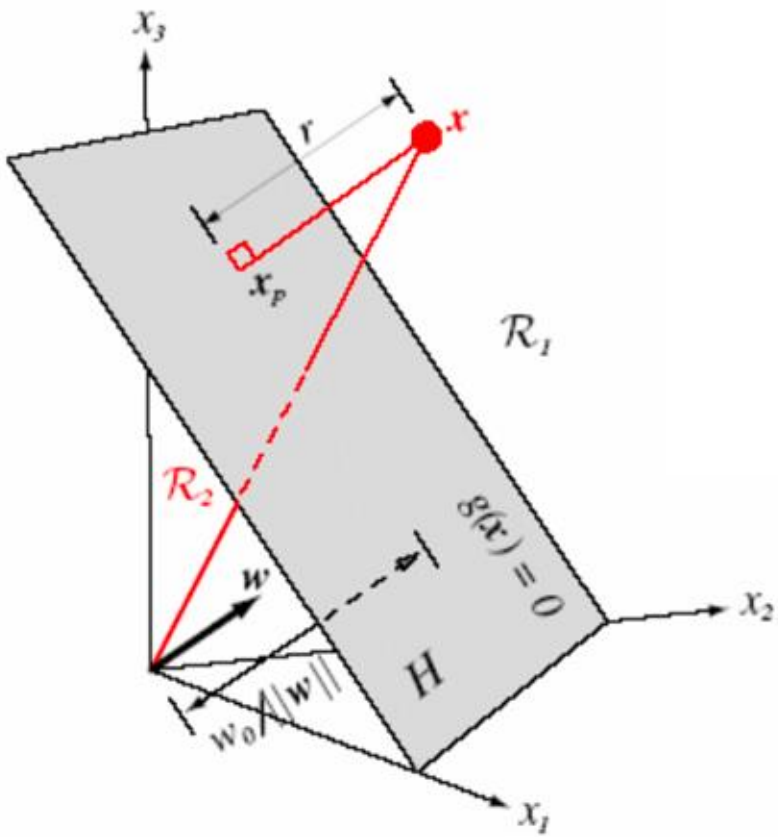
Linear classification problem

- Find a line that separates two classes
- Many solutions exist!
- Which one is the best?



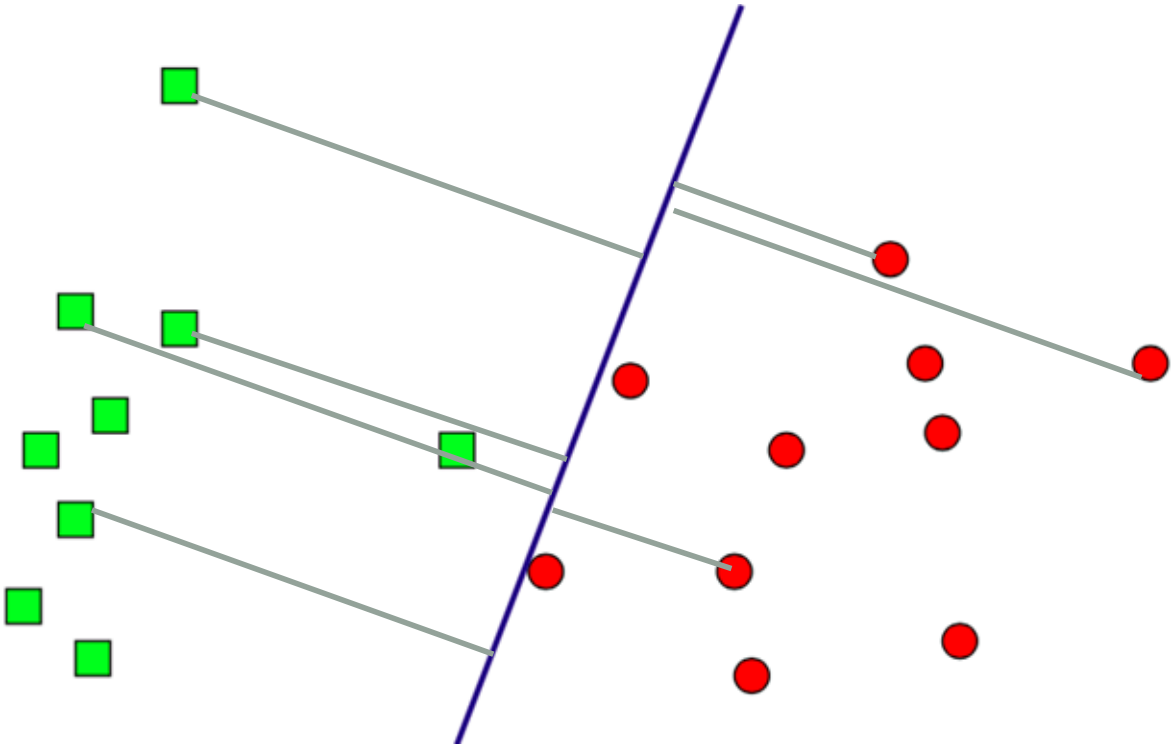
Geometric interpretation of a decision boundary

- Recall a linear classifier (without the logistic part)
  - $g(x) = \mathbf{w}^T\mathbf{x} + w_0$
- If $\mathbf{x}_1$ and $\mathbf{x}_2$ is on the decision boundary, then
  - $\mathbf{w}^T\mathbf{x_1} + w_0 = \mathbf{w}^T\mathbf{x_2} + w_0 = 0$
  - $w^T(\mathbf{x_1}-\mathbf{x_2}) = 0$
    - w is normal to any vector lying in the decision boundary
    - w is normal to the decision boundary hyperplane
    - If $w_0 = 0$, the hyperplane passes through the origin
- Note we can scale w and w_0 without affecting the hyperplane

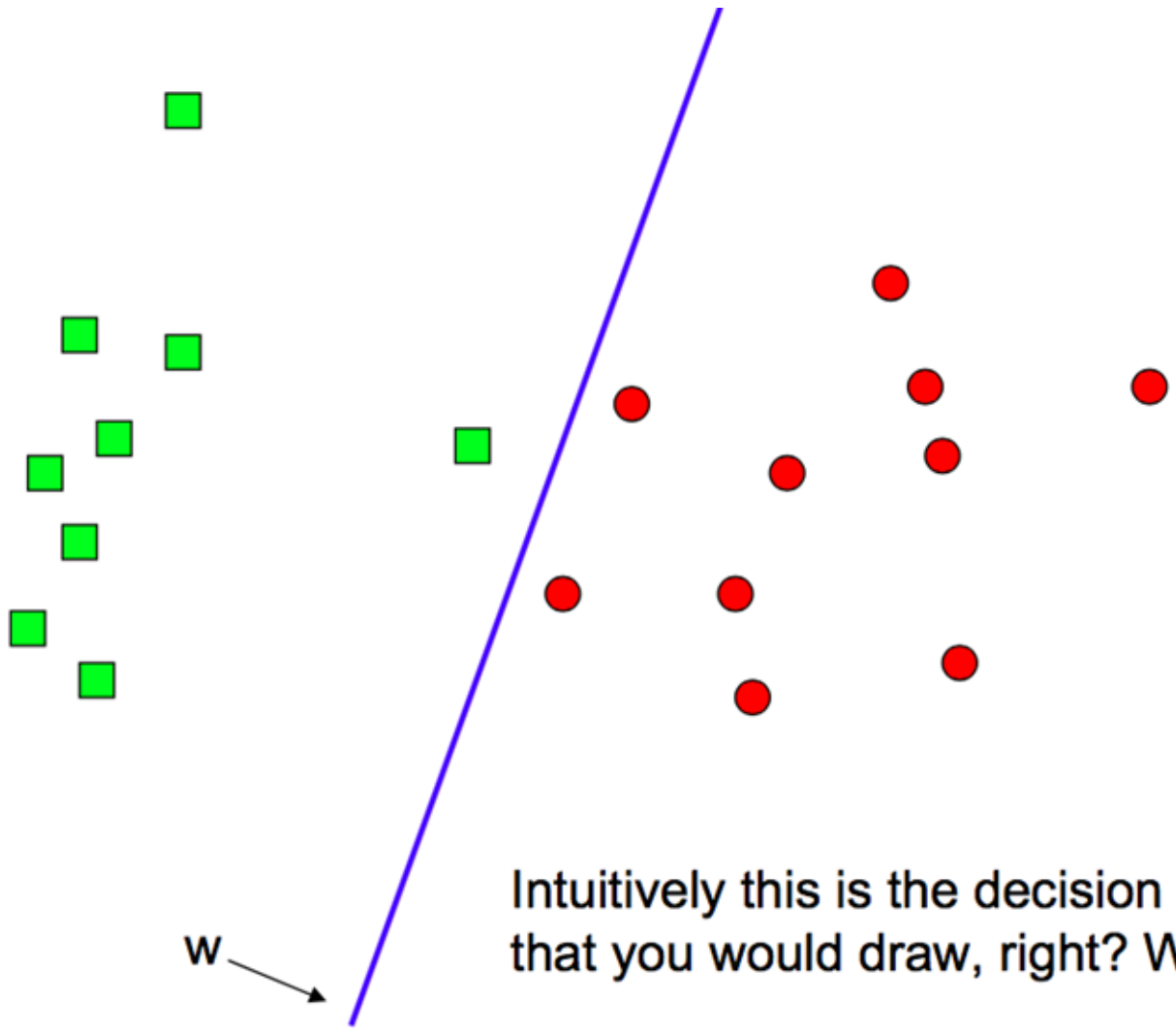


Logistic Regression

- Minimizes sum of L2 distance (square error) between all points to the line



Support vectors

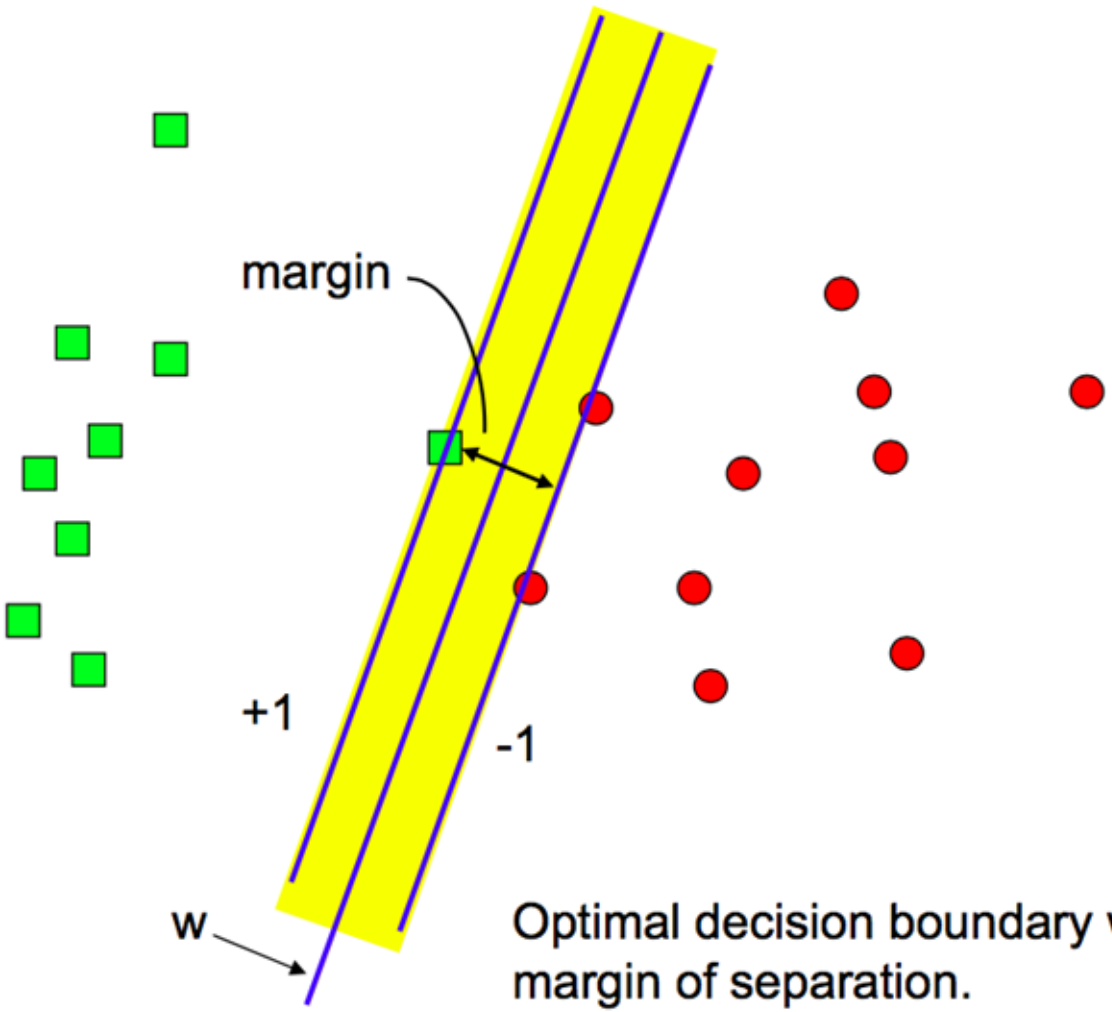


Intuitively this is the decision boundary that you would draw, right? Why?

Support Vector Machines (SVM)

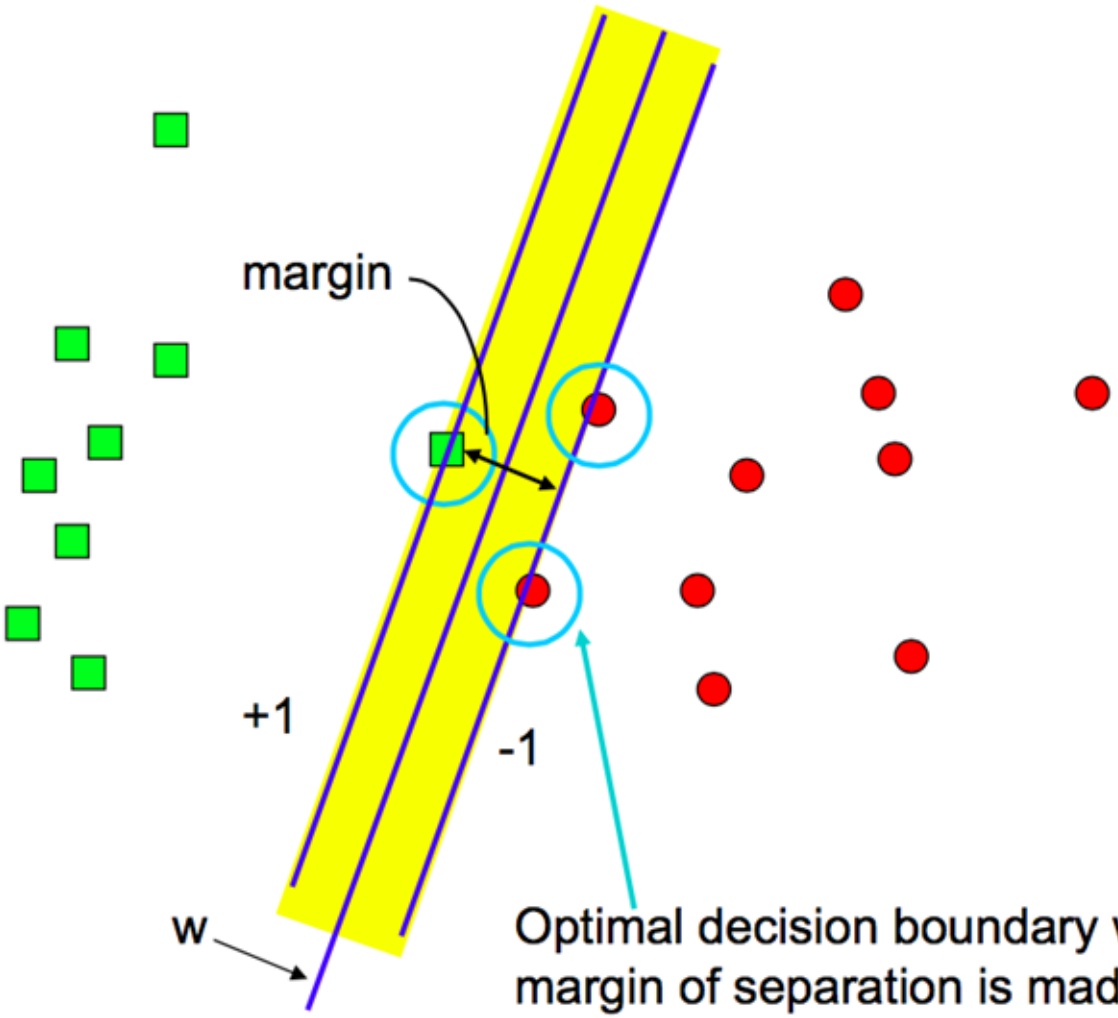
- Goal: improve generalization!
  - Care more about reducing classifier variance than reducing classifier bias
- How?
- Find the decision boundary that gives the most “slack” in classification
  - Don’t care about easy cases, care about borderline cases!
    - Focus on the margin
  - Maximize the “margin of error” between two classes

Support Vectors



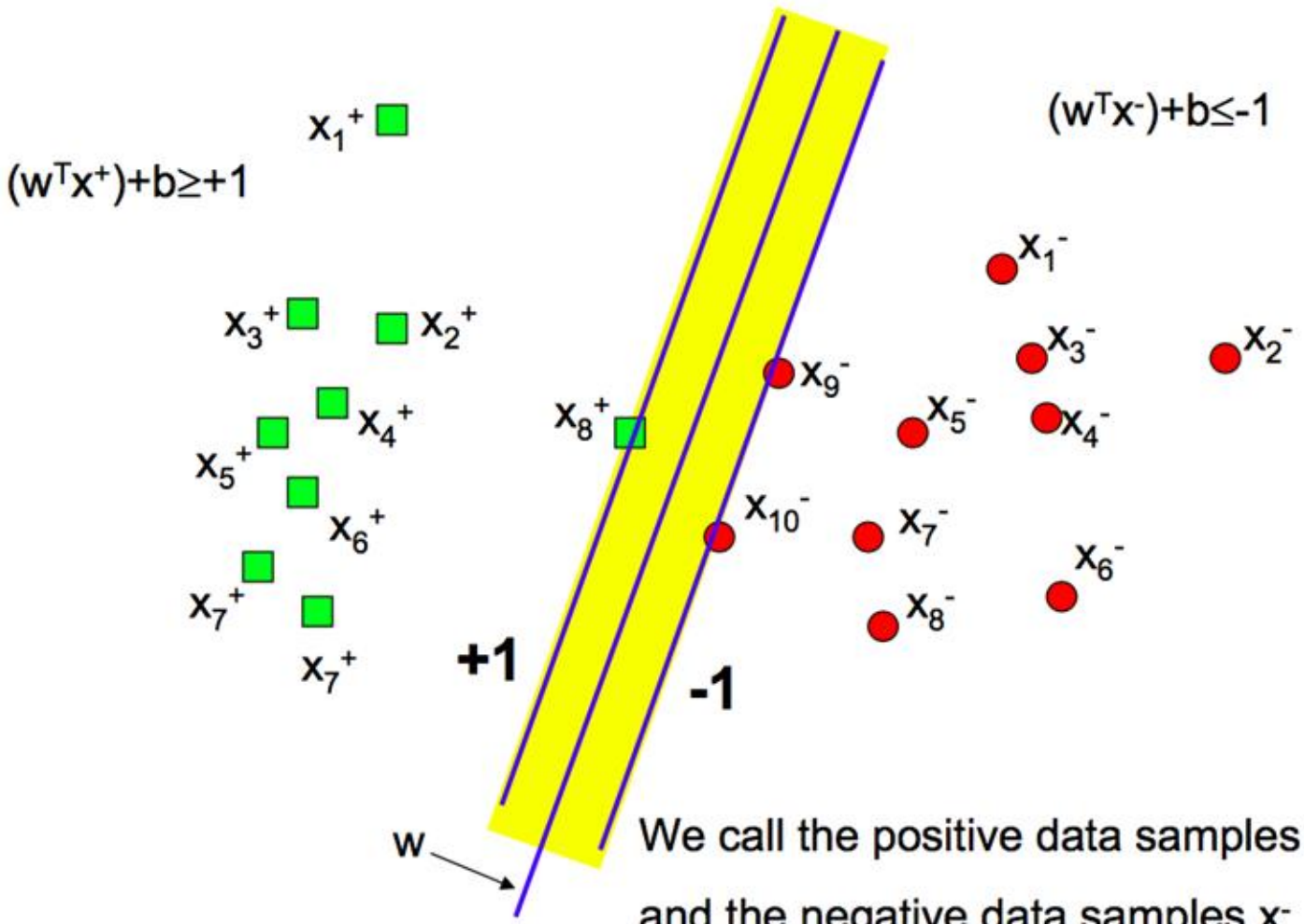
Optimal decision boundary with maximum margin of separation.

Support Vectors



Optimal decision boundary with maximum margin of separation is made from the support vectors (that *support* this margin)

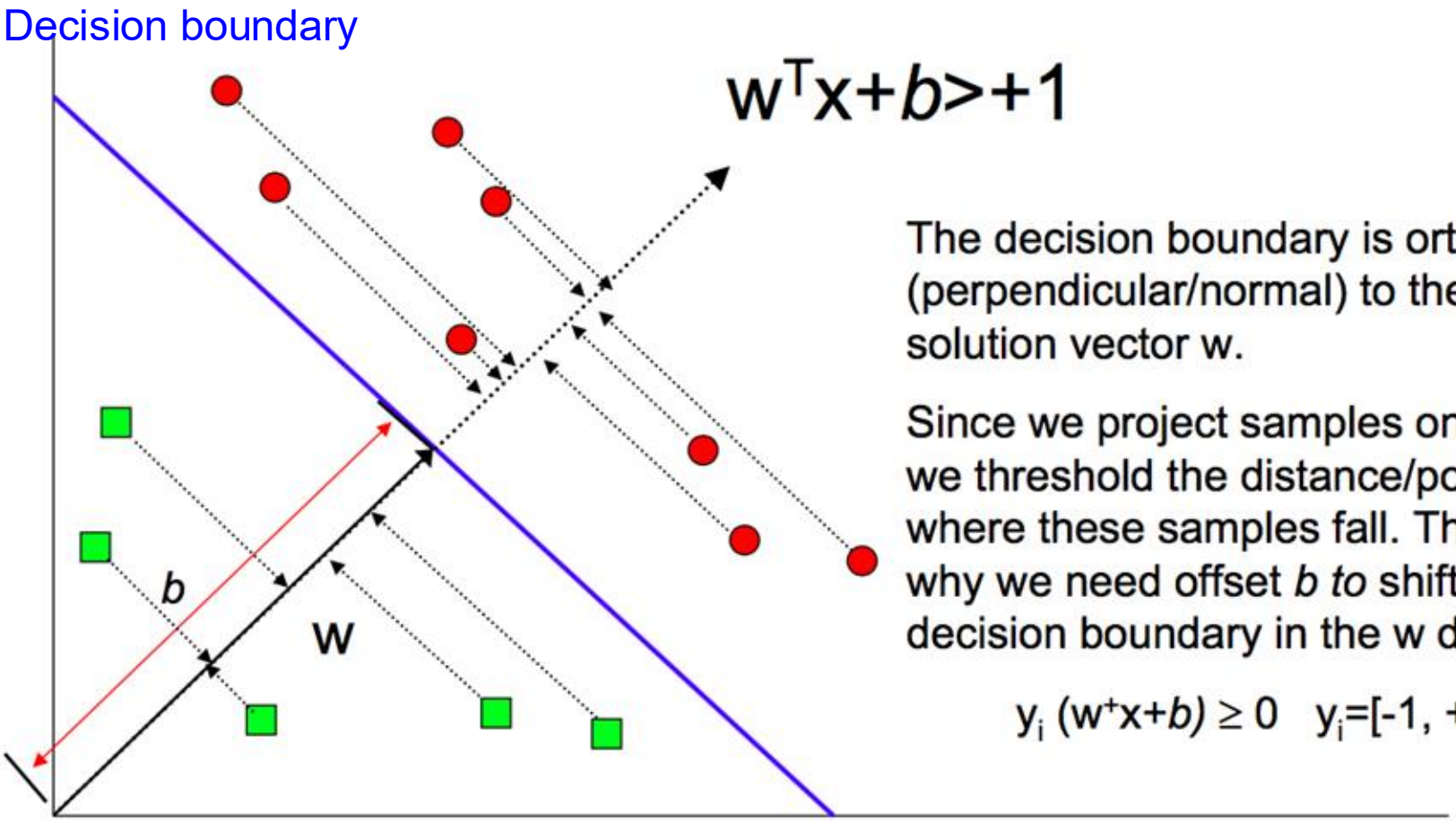
Support Vectors



We call the positive data samples x^+ i.e. $(w^T x^+)+b \geq +1$

and the negative data samples x^- i.e. $(w^T x^-)+b \leq -1$

Geometric interpretation

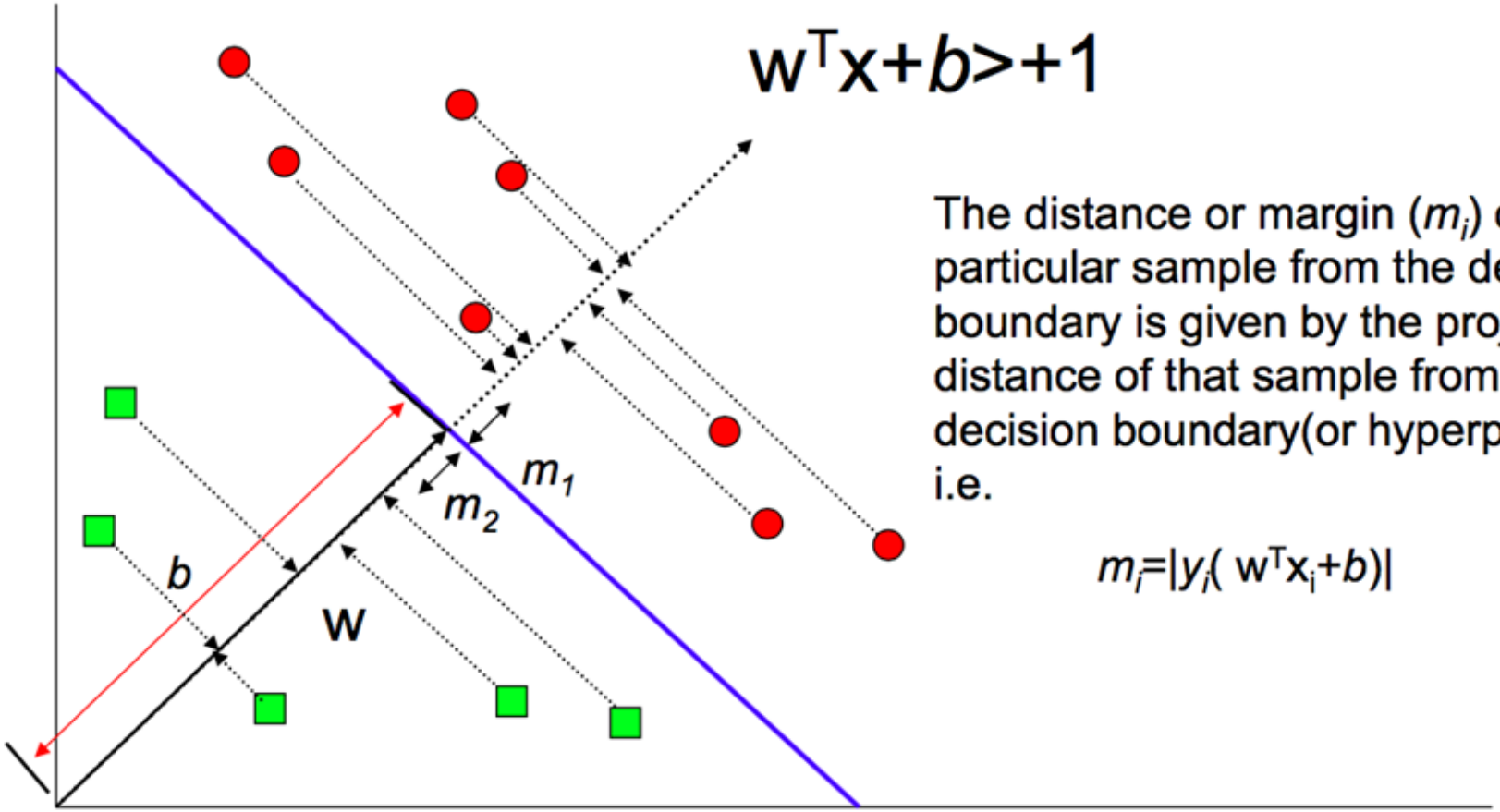


The decision boundary is orthogonal (perpendicular/normal) to the solution vector w.

Since we project samples onto w, we threshold the distance/position where these samples fall. That is why we need offset *b to* shift the decision boundary in the w direction.

$y_i\,(w^+x+b) \geq 0 \quad y_i=[-1, +1]$

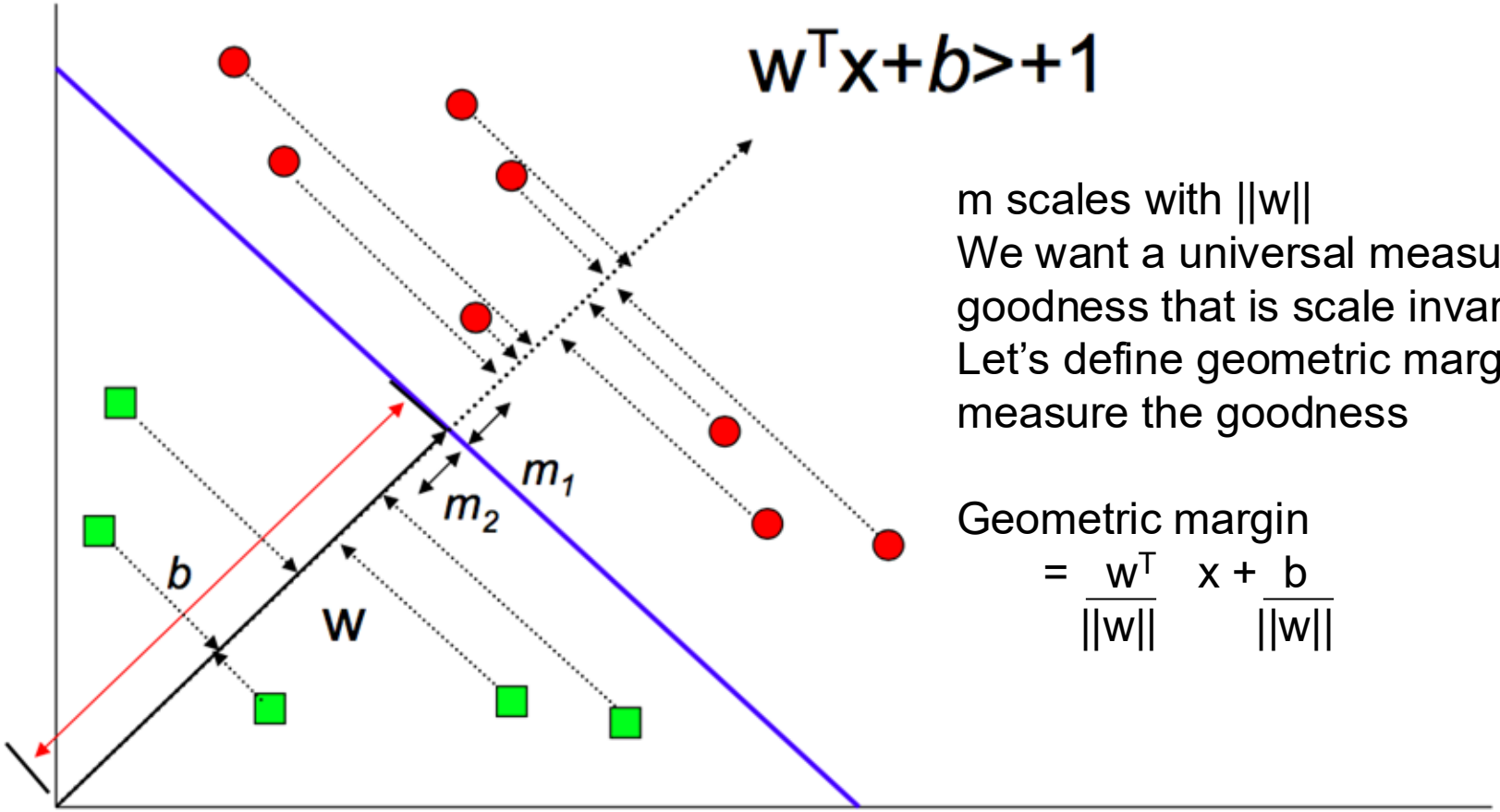
Functional Margin



The distance or margin (m_i) of a particular sample from the decision boundary is given by the projected distance of that sample from the decision boundary(or hyperplane) i.e.

$$m_i=|y_i(w^T x_i+b)|$$

Geometric Margin

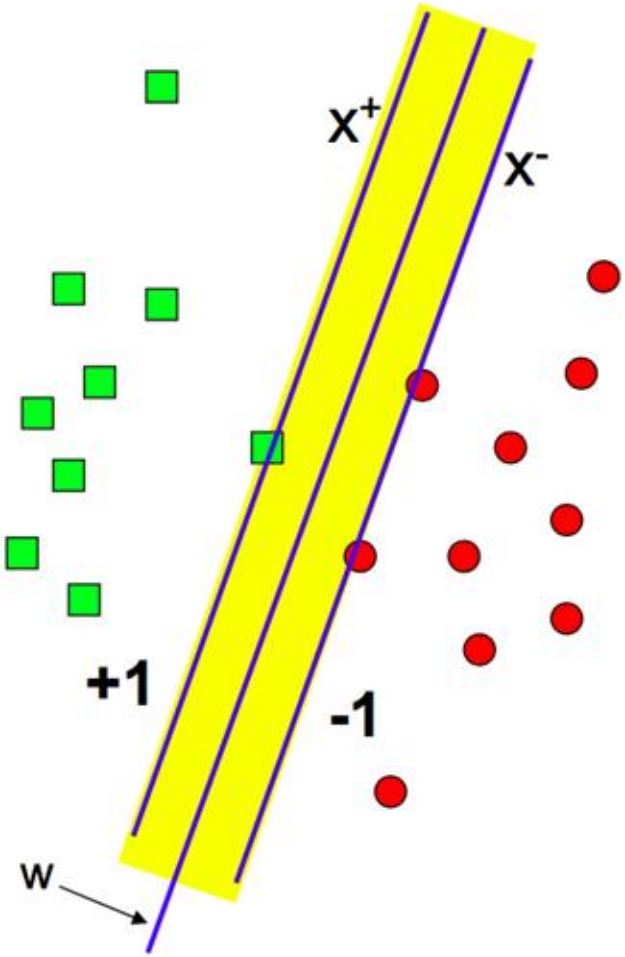


m scales with ||w||
We want a universal measure of goodness that is scale invariant. Let's define geometric margin to measure the goodness

Geometric margin

$$= \frac{w^T}{||w||} x + \frac{b}{||w||}$$

Support Vectors



Let x^+ denote a positive point with functional margin of 1 and x^- denote a negative point respectively.

This implies:

$$w^T x^+ + b = +1$$

$$w^T x^- + b = -1$$

The difference between the two geometric margins is

$$m = \left(\left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x}^+ \right\rangle - \left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x}^- \right\rangle \right)$$

$$= \frac{1}{\|\mathbf{w}\|} \left(\langle \mathbf{w}, \mathbf{x}^+ \rangle - \langle \mathbf{w}, \mathbf{x}^- \rangle \right)$$

$$= \frac{2}{\|\mathbf{w}\|}$$

< > denotes dot product

Max margin

- We want to maximize the margin
  - Maximize $\frac{2}{\|\mathbf{w}\|}$
- Same as minimize $\langle \mathbf{w}, \mathbf{w} \rangle = \mathbf{w}^{\mathrm{T}}\mathbf{w}$

SVM objective function

- Minimize $\mathbf{w}^T\mathbf{w}$
- Subject to
  - $y_i(<\mathbf{w}, \mathbf{x}_i> + b) = y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

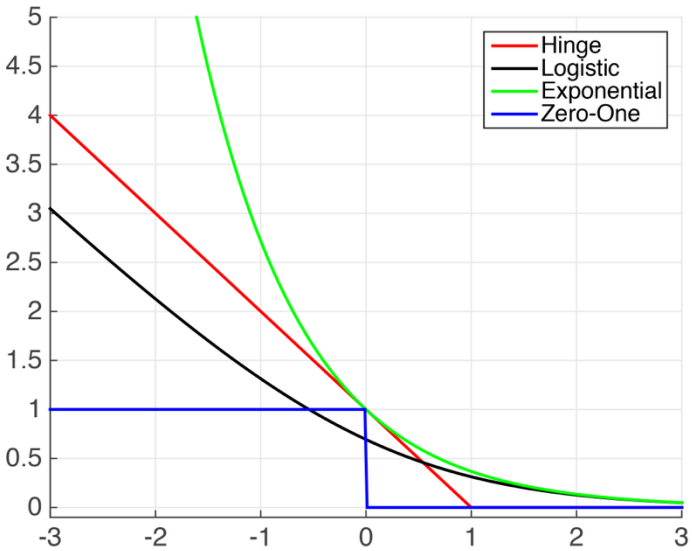
- $y_i = \{+1,-1\}$ depending on the binary class
  - Positive class must fall on the positive side of the boundary
  - Negative class must fall on the negative size

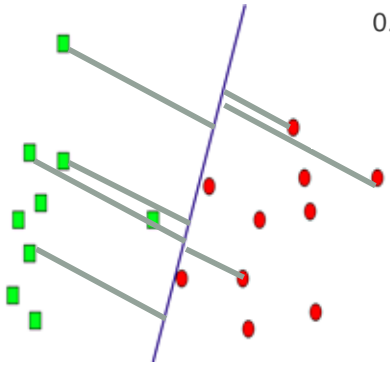
- Convex optimization (No local minimas)
- Can be solved by Quadratic Programing (QP)

Notes on the Losses

- Logistic regression optimizes for the log loss (binary cross entropy loss) https://www.analyticsvidhya.com/blog/2020/11/binary-cross-entropy-aka-log-loss-the-cost-function-used-in-logistic-regression

- SVM optimize for the hinge loss
  - $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$
  - Or $0 \geq 1-y_i(\mathbf{w}^T\mathbf{x}_i + b)$
  - We don't want this inequality to be broken so our effective loss is
    - $\max(0, 1-y_i(\mathbf{w}^T\mathbf{x}_i + b))$

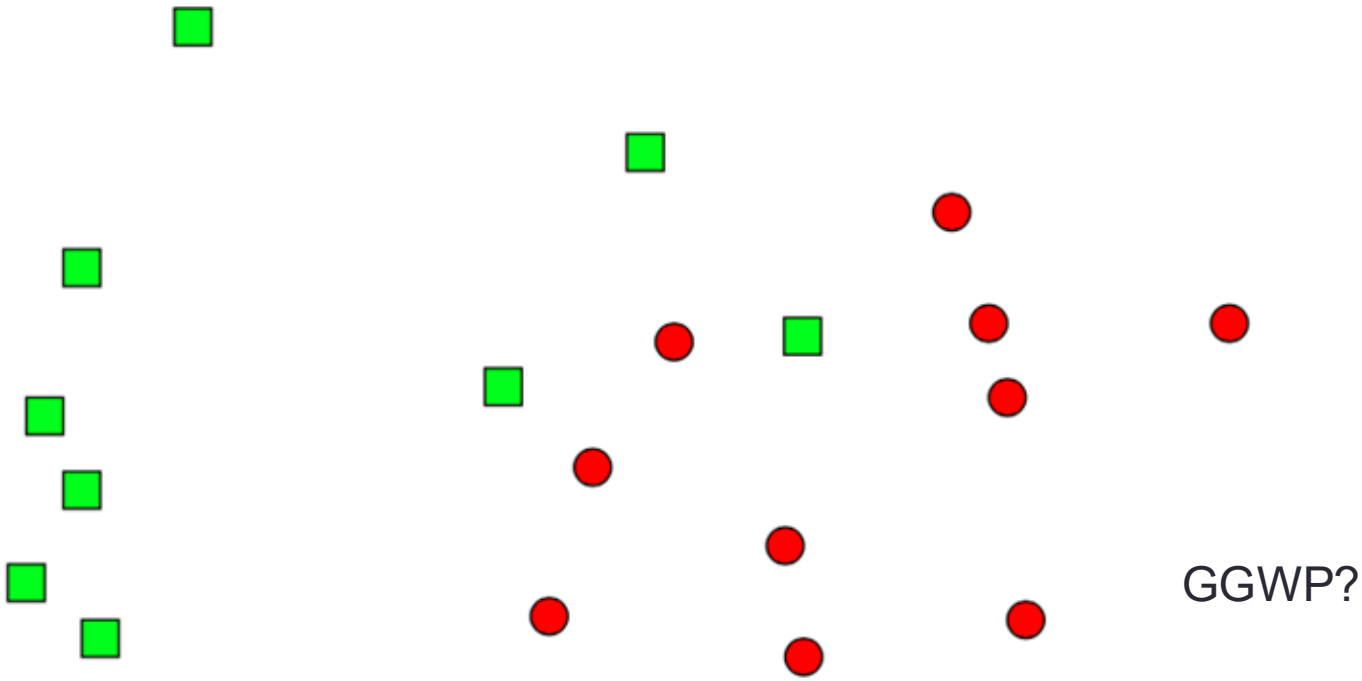




https://www.cs.cornell.edu/courses/cs4780/2017sp/lectures/lecturenote10.html

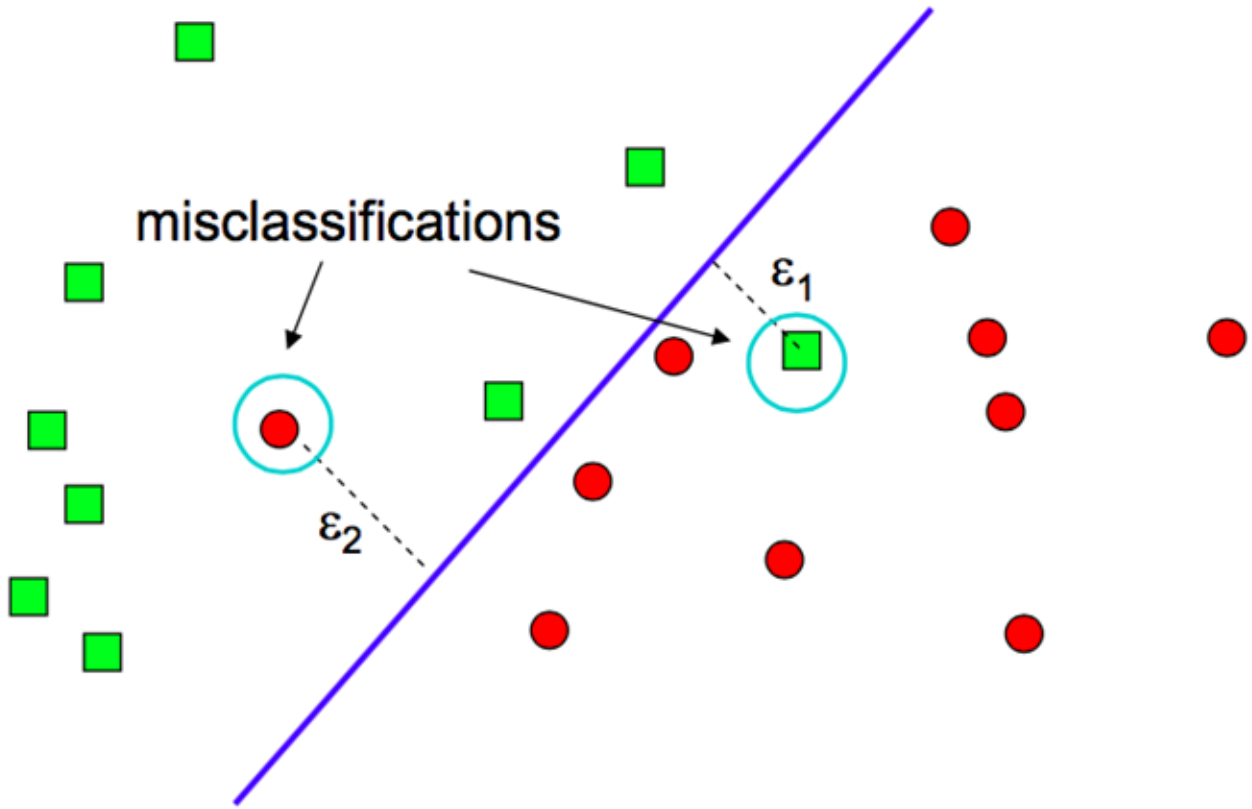
Linearly non-separable

- What happens when you cannot separate the two classes with a linear boundary



Introducing an error term ε

- Aim for a hyperplane that tries to maximize the margin while minimize total error $\Sigma\varepsilon_i$



Slack variables

- We call these error terms “Slack variables”
- Give SVM some slack so that the SVM can do its job.

SVM objective function

- Minimize $\mathbf{w}^T\mathbf{w}$
- Subject to
  - $y_i(<\mathbf{w}, \mathbf{x}_i> + b) = y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- $y_i = \{+1,-1\}$ depending on the binary class
  - Positive class must fall on the positive side of the boundary
  - Negative class must fall on the negative size

SVM objective with slack

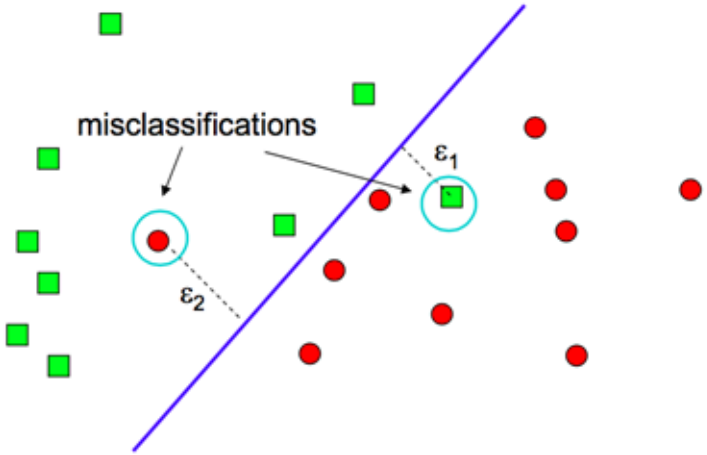
- Minimize $\mathbf{w}^T\mathbf{w} + C\Sigma\varepsilon_i$
- Subject to

C is a weight parameter, how much we care about slack

$$\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b \geq 1 - \varepsilon_i \qquad for \quad +ve \quad class$$

$$\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b \leq -1 + \varepsilon_i \qquad for \quad -ve \quad class$$

$$\varepsilon_i \geq 0 \qquad \forall i$$



Primal form – Dual form

- In optimization, many problems can be framed in two ways
  - Original version: Primal form
  - Transformed version: Dual form
- Both yield the same solution (under some conditions), but sometimes solving one method is a lot easier than the other.

SVM objective function – Primal form

- Minimize $\mathbf{w}^T\mathbf{w}$
- Subject to
  - $y_i(<\mathbf{w}, \mathbf{x}_i> + b) = y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- $y_i = \{+1,-1\}$ depending on the binary class
  - Positive class must fall on the positive side of the boundary
  - Negative class must fall on the negative size

Primal Lagrangian Form

$$L(\mathbf{w},b,\alpha) = \tfrac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w} - \sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b) - 1\right]$$

margin

constraints

- Where α_i are the lagrange multipliers $\alpha_i \geq 0$
- We want to optimize this function

Primal form **w**

- Primal form:

$$L(\mathbf{w},b,\alpha)=\tfrac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}-\sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x}_i+b)-1\right]$$

- Differentiate with respect w to find

$$\frac{L(\mathbf{w},b,\alpha)}{\partial\mathbf{w}}=\mathbf{w}-\sum_{i=1}^{N}\alpha_i y_i\mathbf{x}_i=\mathbf{0}$$

$$\mathbf{w}=\sum_{i=1}^{N}\alpha_i y_i\mathbf{x}_i$$

Primal form b

- Primal form:

$$L(\mathbf{w},b,\alpha)=\tfrac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}-\sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x}_i+b)-1\right]$$

- Differentiate with respect b to find

$$\frac{L(\mathbf{w},b,\alpha)}{\partial b}=\sum_{i=1}^{N}\alpha_i y_i=0$$

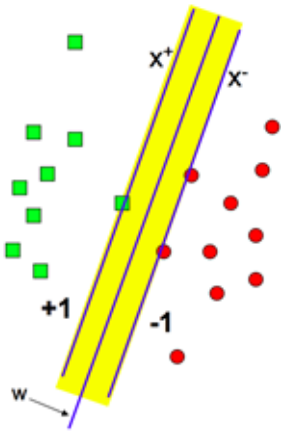
$$\sum_{i=1}^{N}\alpha_i y_i=0$$

Primal form solution

- w is a linear combination of our training data

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i \qquad \sum_{i=1}^{N} \alpha_i y_i = 0 \qquad \alpha_i \geq 0$$

- Which training data depends on whether that training data is a support vector (the vector on the boundary) or not



α_i will be 0 for non support vectors

Dual form

- Primal form:

$$L(\mathbf{w},b,\alpha)=\tfrac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}-\sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x}_i+b)-1\right]$$

- Substitute $\mathbf{w}=\sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i$ back into above Eq.

$$L(\mathbf{w},b,\alpha)=\tfrac{1}{2}\mathbf{w}^{\mathbf{T}}\mathbf{w}-\sum_{i=1}^{N}\alpha_i\left[y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x}_i+b)-1\right]$$

$$=\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}y_i y_j \alpha_i \alpha_j \left\langle x_i, x_j\right\rangle-\sum_{i=1}^{N}\sum_{j=1}^{N}y_i y_j \alpha_i \alpha_j \left\langle x_i, x_j\right\rangle+\sum_{i=1}^{N}\alpha_i$$

$$\boxed{=\sum_{i=1}^{N}\alpha_i-\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}y_i y_j \alpha_i \alpha_j \left\langle x_i, x_j\right\rangle}$$

Dual form

Dual form optimization

- Dual form:

$$\sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} y_i y_j \alpha_i \alpha_j \left\langle x_i, x_j \right\rangle$$

- Subject to the constraints $\sum_{i=1}^{N} \alpha_i y_i = \mathbf{0}$ and $\alpha_i \geq 0$

- Again this is solvable with QP.

What does the dual form give us?

- Dual form

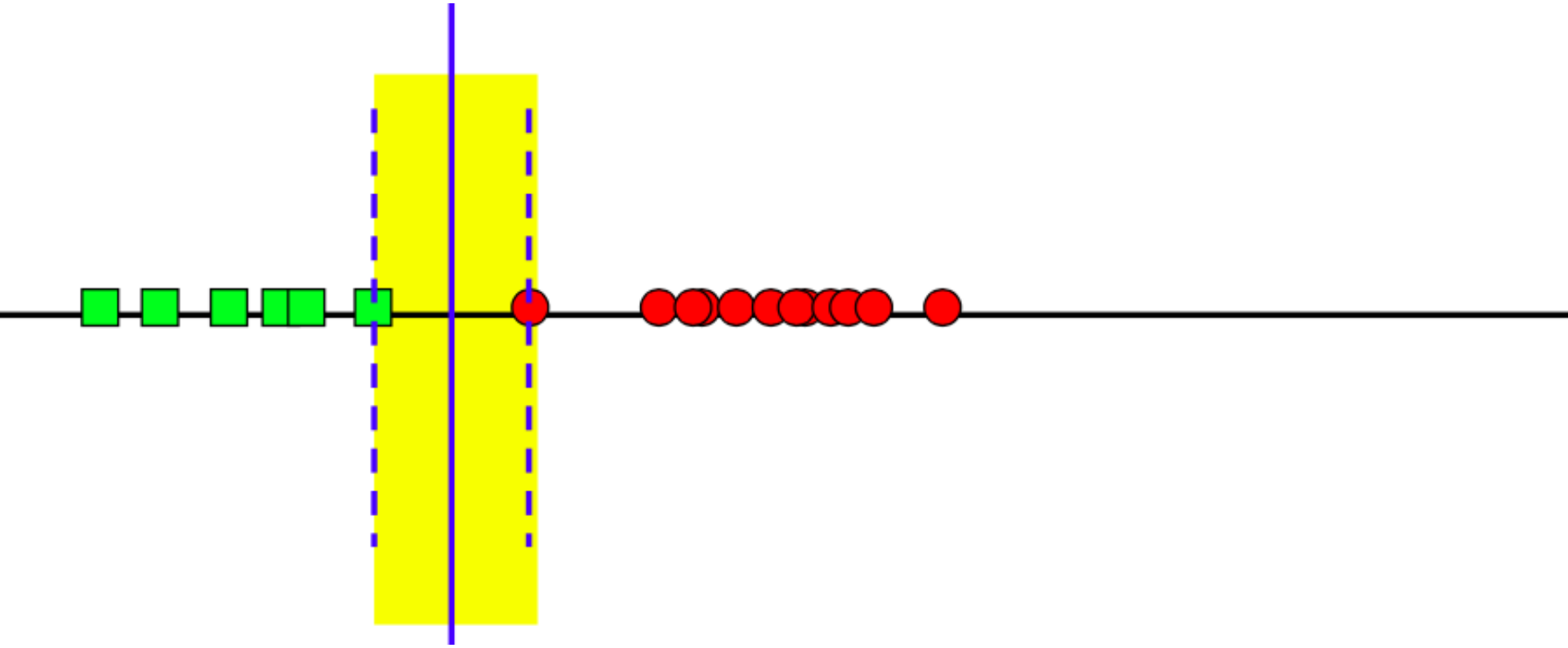
$$\sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} y_i y_j \alpha_i \alpha_j \langle x_i, x_j \rangle$$

- Optimize using pairwise inner product of inputs instead of inputs
- Gram matrix (matrix of inner product between inputs)

- How is this useful?

Example SVMs

- Easy



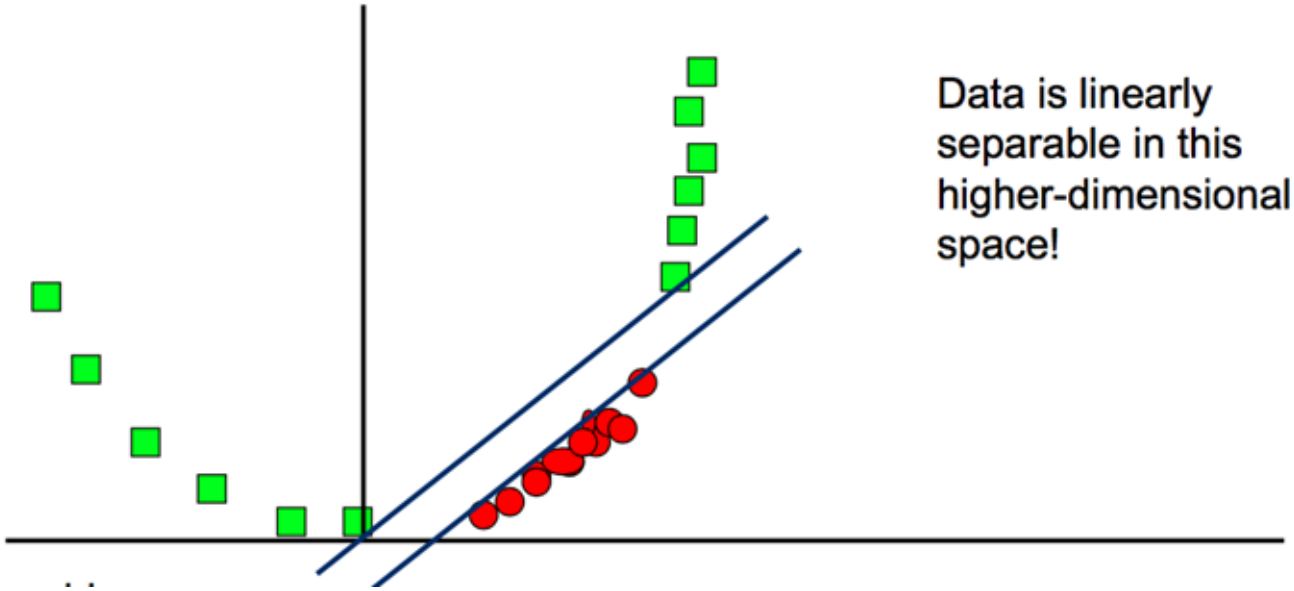
Example SVMs

- ??????

Adding features (non-linear transformation)

- Remember we add non-linear features to linear regression to do non-linear fitting
- Consider as a non-linear transformation to higher dimensional space

$F(x) \rightarrow (x, x^2)$



What about curse of dimensionality?

- Didn't we say higher dimension sucks?

- In this case our data is NOT separable in the original space, so we want to map to higher dimensions
  - Just high enough so the data is separable

- However, this will require higher compute because of dimensionality
  - Dual form will help with this!

Mapping functions

$$\phi : X \rightarrow F$$

- A mapping function that maps to higher dimensional space
- Our solutions become

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \Phi(\mathbf{x}_i)$$

- And if we want to classify a new sample

$$\mathbf{w}^{\mathrm{T}} \Phi(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i y_i \left\langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}\ ) \right\rangle$$

Mapping function dual form

- In the dual form we solve

$$\sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} y_i y_j \alpha_i \alpha_j \left\langle \Phi(x_i), \Phi(x_j) \right\rangle$$

Inner product of the higher space

- Claim: sometimes inner product of the higher space can be solved directly without mapping to the higher space and compute the inner product

Kernel function

- We define the inner product in the mapped space as a kernel function K(x,y)

$$K(\mathbf{x},\mathbf{y}) = \langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle$$

- Kernel of x -> (x, x^2)
  - $xy + x^2y^2$
- Kernel of x -> (x, x^2,x^3)
  - $xy + x^2y^2+x^3y^3$

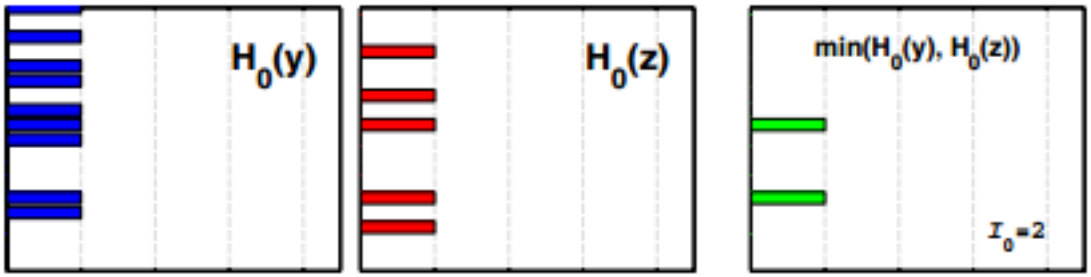
Kernel functions

- Sometimes we don't even know what the mapping function is, but we "dream up" a kernel
  - A kernel is legitimate if it satisfies "Mercer's Condition"
  - Mercer's condition guarantees existence of a higher dimensional space that yields the dot product, but we just don't know what space

Histogram intersection kernels

- Given input features which are histograms
  - Histogram of first data $H_0(y)$. Histogram of second data $H_1(z)$
- The Kernel that counts the intersection of the histograms is a valid kernel.
  - E.g. Sum of $\min(H_0(y), H_1(z))$ for all histogram bins

- (One of the most used kernels in computer vision)



Radial Basis Kernels

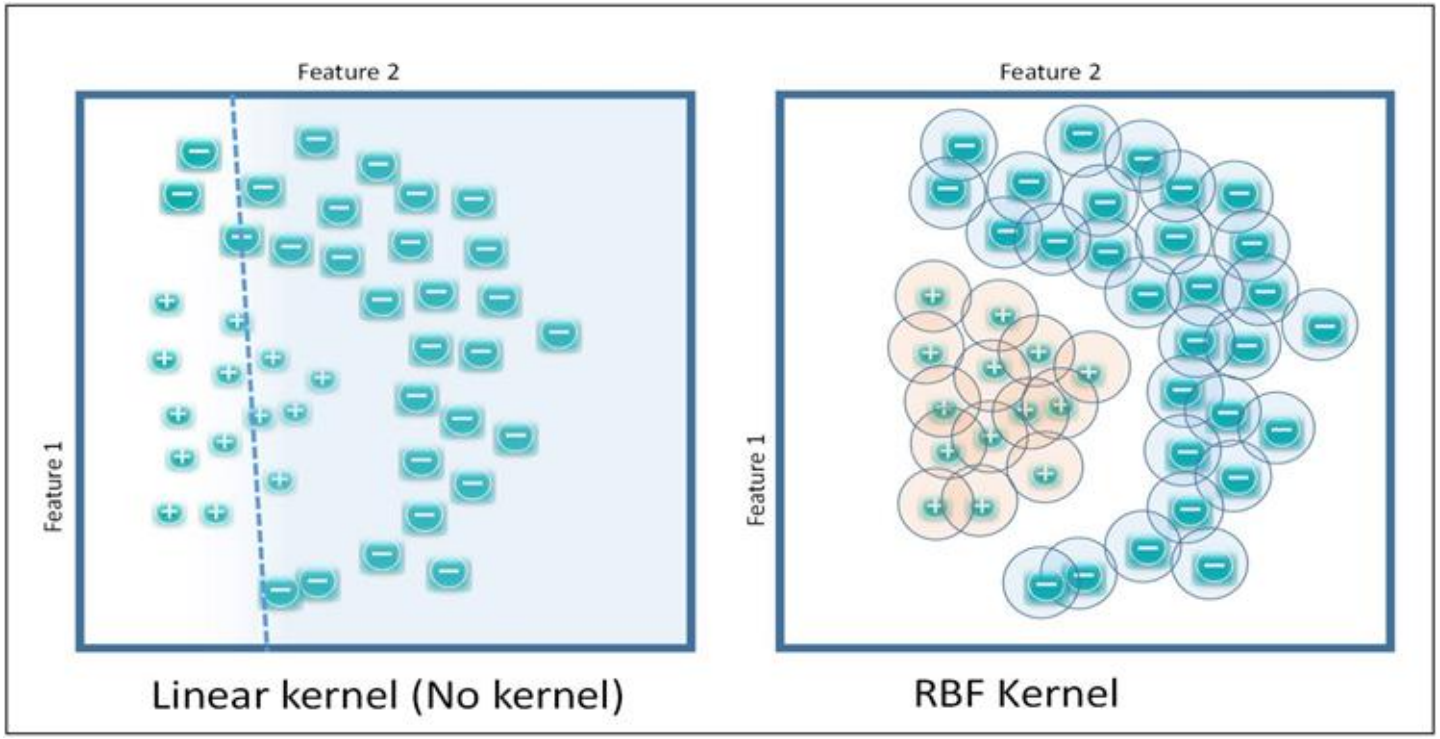
- Most powerful general-purpose kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

- Pretty much a Gaussian with mean x' and variance σ^2
  - Variance is a parameter to select
- This kernel comes from a space that has infinite dimensions

RBF kernels

- Think of RBF as putting Gaussians onto the support vectors



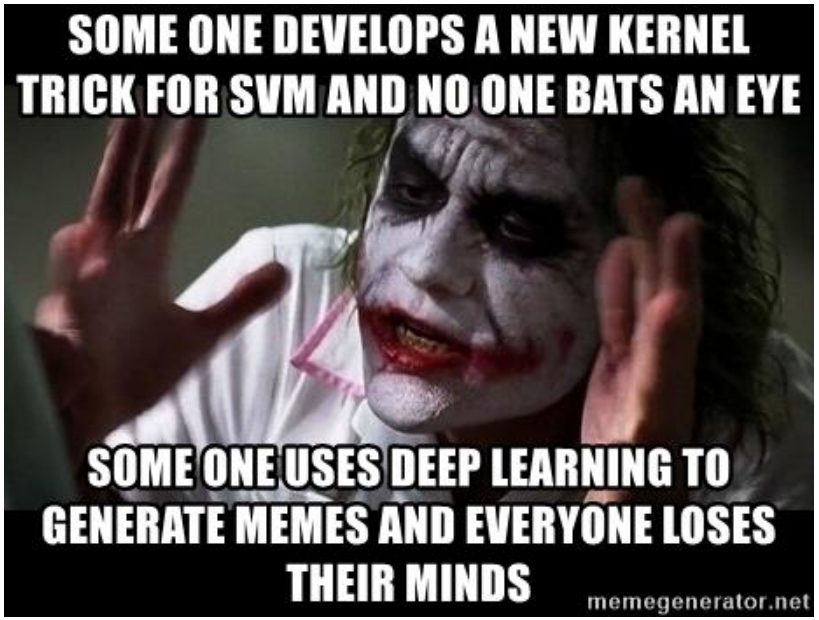
https://www.intechopen.com/books/epilepsy-topics/nonlinear-epilepsy-forewarning-by-support-vector-machines

Design your own kernel

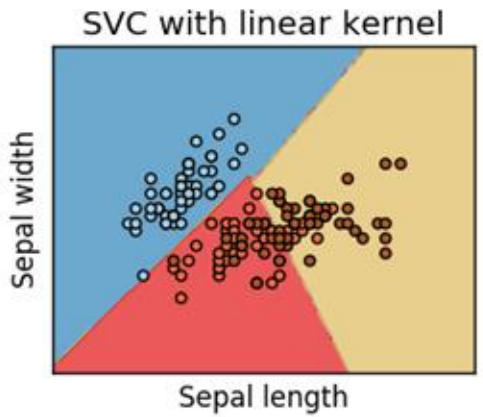
- A kernel is valid if (Mercer's condition)
  - It's symmetric $K(x,y) = K(y,x)$
  - The matrix of K where $K_{ij} = K(x_i,x_j)$ is positive definite (for any x_i, x_j)

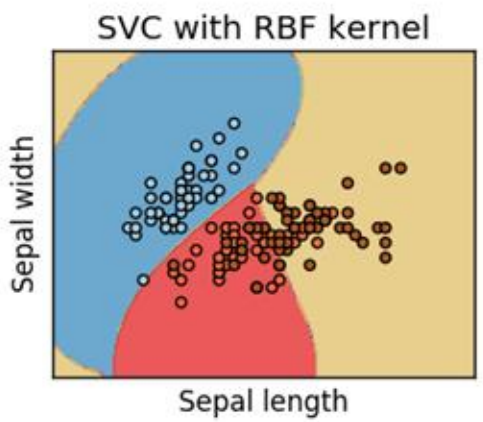
- Build from existing kernels
  - If K1 K2 are valid kernels
    - K = aK1 + bK2
    - K = K1*K2
    - K = K1^(K2)

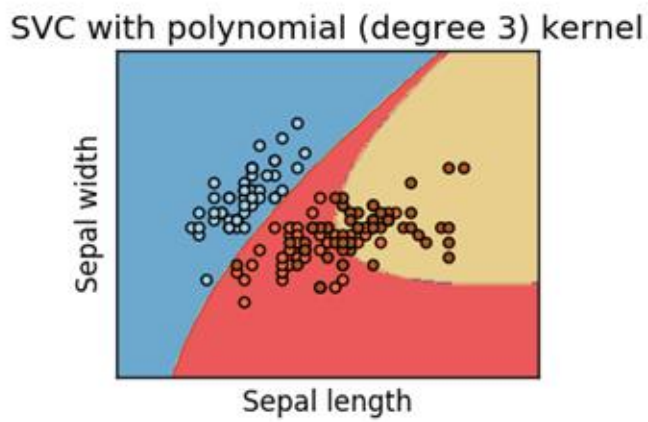
    are valid kernels



SVM examples







Think of SVM with RBF as K-NN with “support vectors”

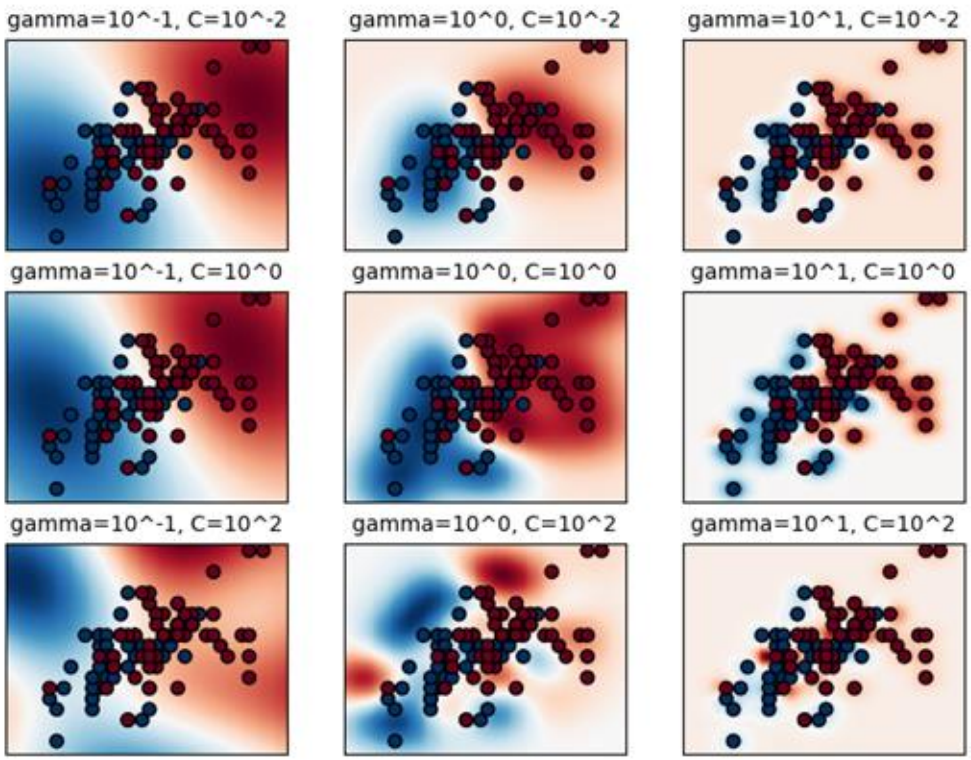
http://lijiancheng0614.github.io/scikit-learn/modules/svm.html

RBF SVM and sci-kit learn

- Gamma is the inverse of the variance
- C is the inverse slack variable weight

Less Gamma
Underfit

Less C
Underfit



http://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

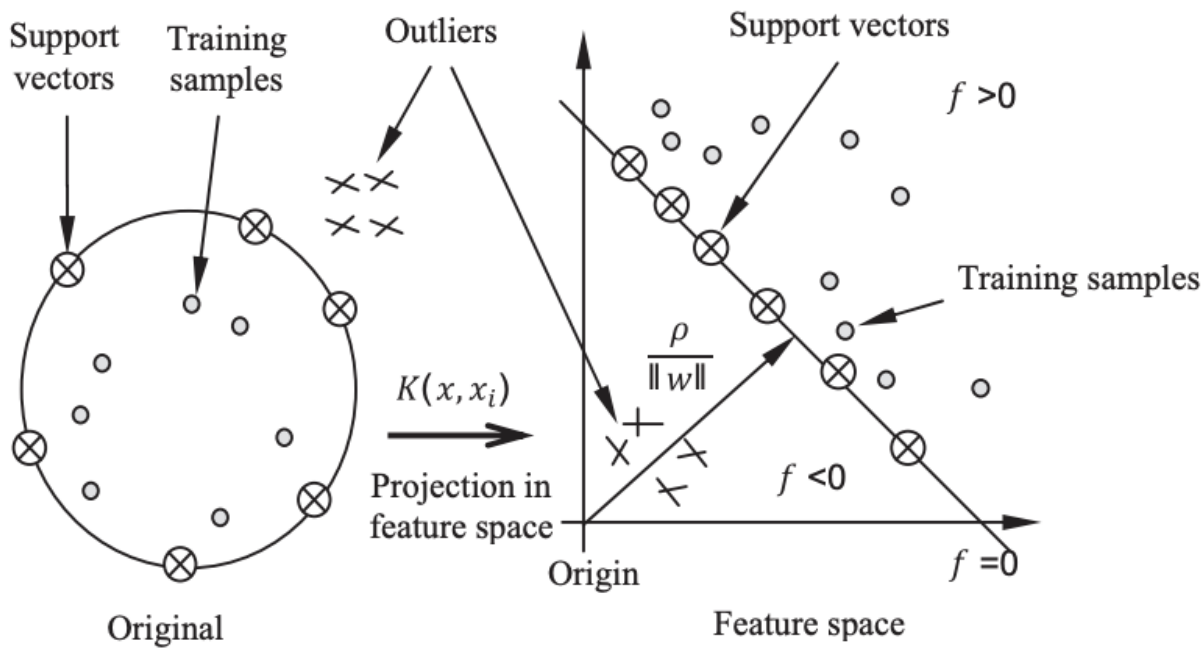
One class SVMs

- Sometimes it is easy to get positive examples but hard to acquire all possible negative examples
  - Email spam filter
    - We kind of know what a good email looks like. And we have lots of examples
    - Hard to model what a spam is. Spammer can change the format and evade detection.

- Solution: train on just the positive class
  - Model what that class looks like
  - Anything that deviates too much from it is considered negative examples

How?

- Separates the data from the “origin” (in mapped space)
- Maximize the distance between data points and the origin



https://www.sciencedirect.com/science/article/abs/pii/S0031320314002751

http://rvlasveld.github.io/blog/2013/07/12/introduction-to-one-class-support-vector-machines/

SVM objective with slack

- Minimize $\mathbf{w}^T\mathbf{w} + C\Sigma\varepsilon_i$
- Subject to

C is a weight parameter, how much we care about slack

$$\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b \geq 1 - \varepsilon_i \qquad for \quad +ve \quad class$$

$$\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b \leq -1 + \varepsilon_i \qquad for \quad -ve \quad class$$

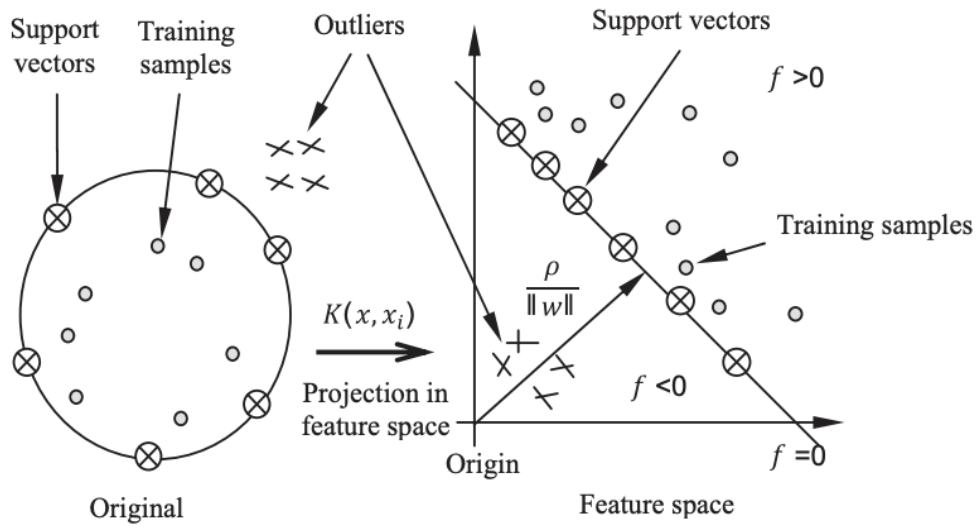
$$\varepsilon_i \geq 0 \qquad \forall i$$

One class SVM with slack

- Minimize $\mathbf{w}^T\mathbf{w} + 1/(vn)\ \Sigma\varepsilon_i - \rho$
- Subject to ρ is the radius of the ball that we also need to optimize

$$\mathbf{w}^{\mathbf{T}}\mathbf{x}_i + b \geq \rho - \varepsilon_i$$

$$\varepsilon_i \geq 0 \qquad \forall i$$



The hyper parameter v (nu, greek letter n) sets the upper bound of the fraction of training examples to be regarded negative (even though we only put in positive examples). Also called nu-svm

SVM Notes

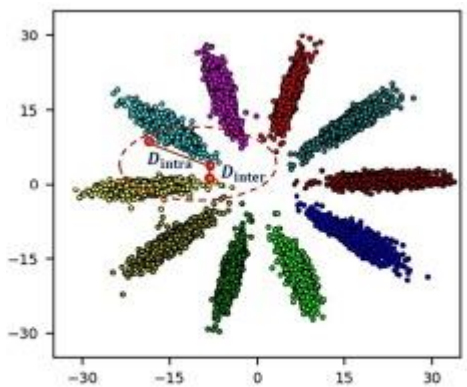
- One of the strongest models and hard to overfit
  - With proper tuning of hyperparameters (kernel, gamma, etc.)
- Convex optimization – no need to try multiple initializations
- Easiest to do one-class setups
- Does not scale well to large datasets
  - Need to compute Gram matrix (sometimes does not fit in memory if too many data points)
  - Inference time and memory requirement during inference depends on the number of support vectors

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \Phi(\mathbf{x}_i)$$

Margin-based today

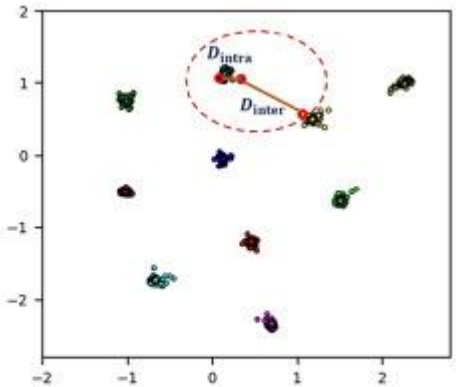
- Many deep learning models now incorporate some kind of margin in the loss function
  - Used in face recognition, embedding-based models

Regular loss without margin



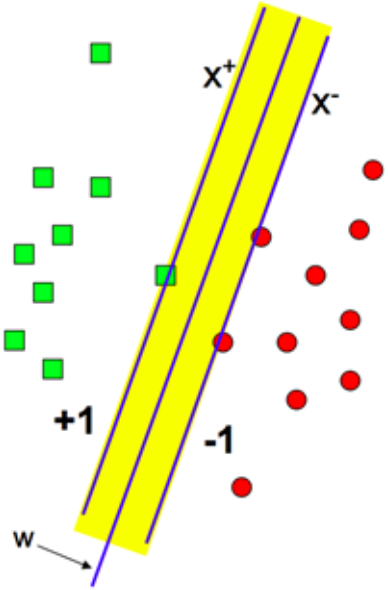
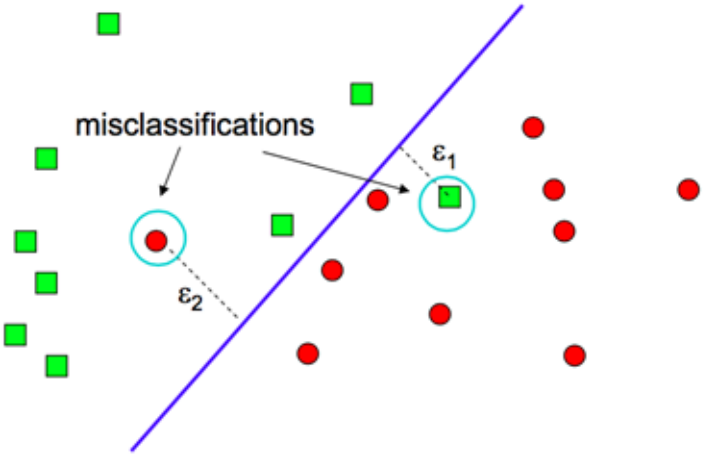
(a) Softmax loss

Loss with margin term



(b) I2CS loss

https://www.sciencedirect.com/science/article/pii/S0925231220318178

Summary

- SVMs
  - Max margin loss
  - Slack
  - Dual-primal
    - Kernel (inner product of higher space)
  - RBF kernels
  - One class SVM